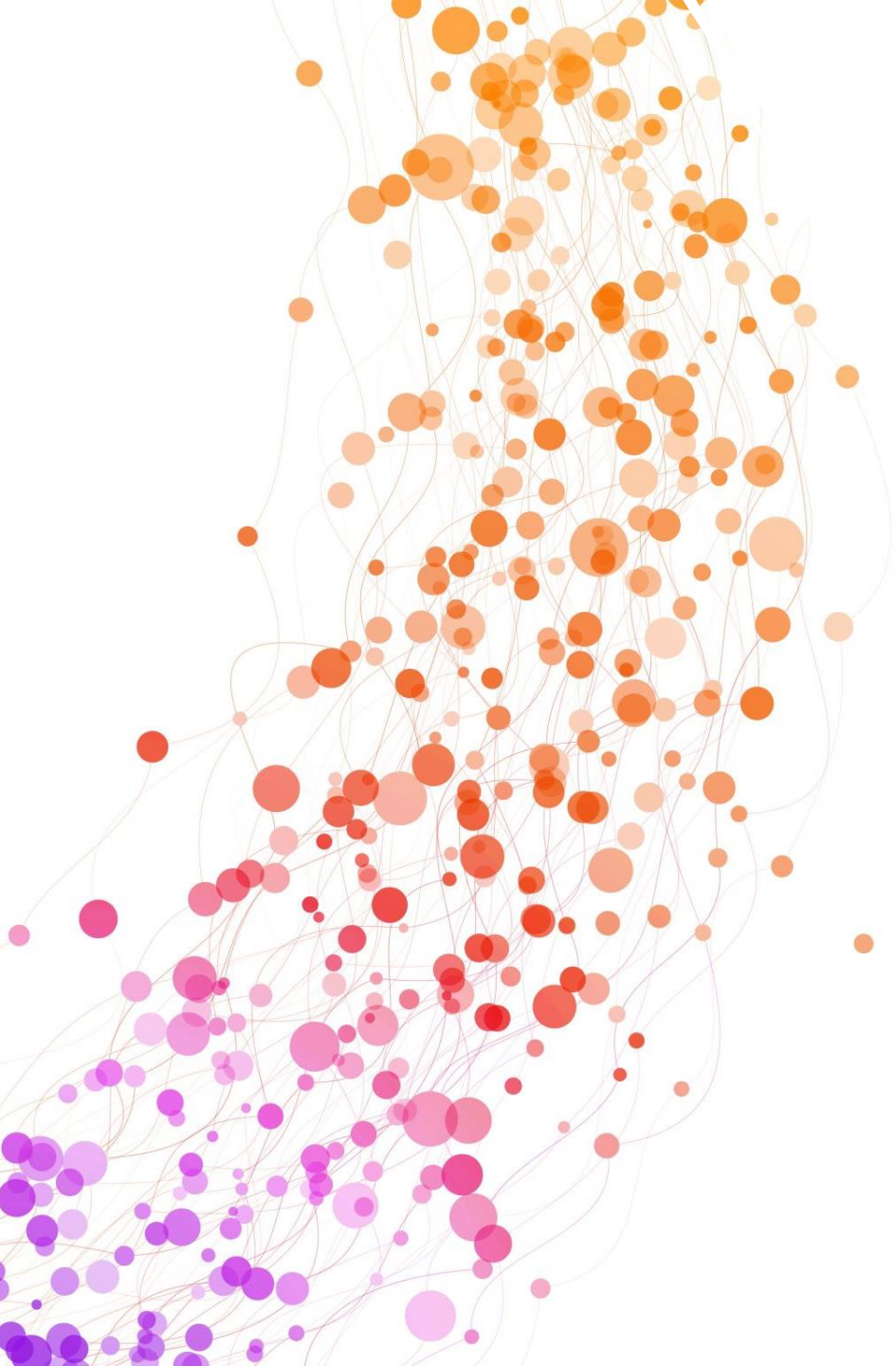


Deep Learning Bootcamp 4

Natural Language Processing

Speakers: Alka Luqman & Tu Ngo (CCDS)

Student Life@GSC Collaborative Events

Bootcamp Code

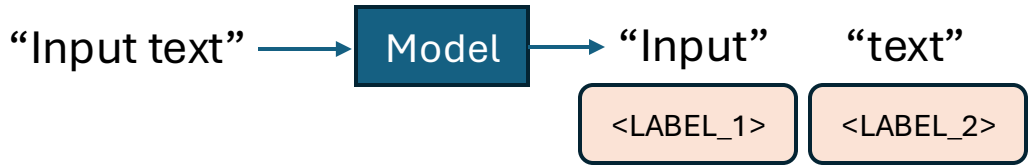
- Website

  https://ntu-dl-bootcamp.github.io/deep-learning-2026/

NLP tasks

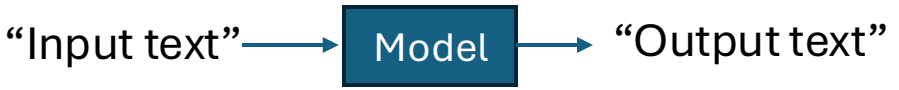
Classification

- Sentiment analysis + −
- Language detection GB IN CN DE ES
- Topic modeling Politics, Tech, Econ…



- Part of speech tagging VRB PRN NN ADJ
- Named entity recognition Person, Location, Time, …
- Dependency parsing

Generation



- Machine translation
- Question answering
- Summarization
- Text generation

Tokenization

- How to represent text?

A teddy bear  in our Bootcamp.

Character-level

A	_	t	e	d	d	y	_	b	e	a	r	_		_	i	n	_	o	u	r	_	B	o	o	t	c	a	m	p	.

Word-level

A	teddy	bear		in	our	Bootcamp	.

Subword-level

A	ted	##dy	bear		in	our	Bootcamp	.

One Hot Encoding

Sentence : This is a cat

	VOCABULARY					
	a	cat	is	NTU	this	good
this	0	0	0	0	1	0
is	0	0	1	0	0	0
a	1	0	0	0	0	0
cat	0	1	0	0	0	0

Let’s assume we have a super tiny vocabulary list:
{“a”, “cat”, “is”, “NTU”, “this”, “good” }

Encoded : [000010 001000 100000 010000]

One Hot Encoding

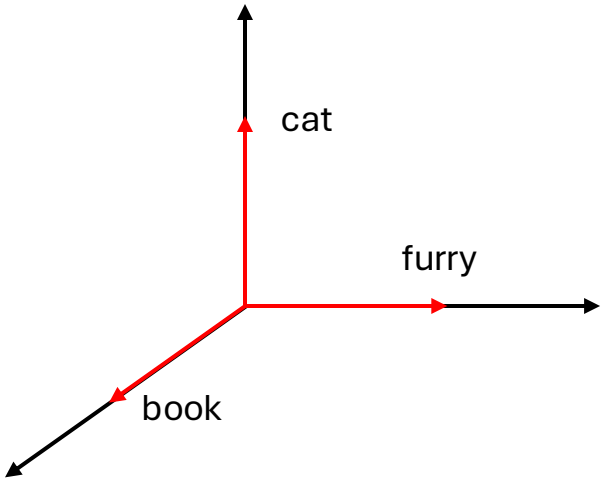
Sentence : This is a cat

Sentence : [000010 001000 100000 010000]

- How long should this vector be? *A whole vocab*
- What if vocabulary is updated? *Update every single vector*
- Vectors are orthogonal *No correlation, next slides*

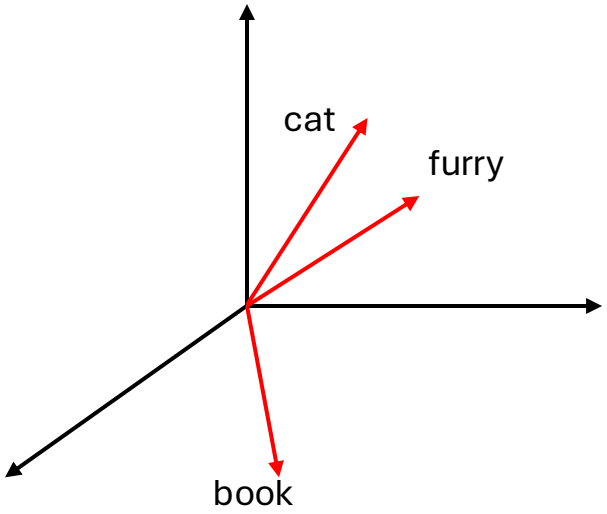
Embeddings

One-hot encoding



Similarity(cat, furry) = 0
Similarity(cat, book) = 0

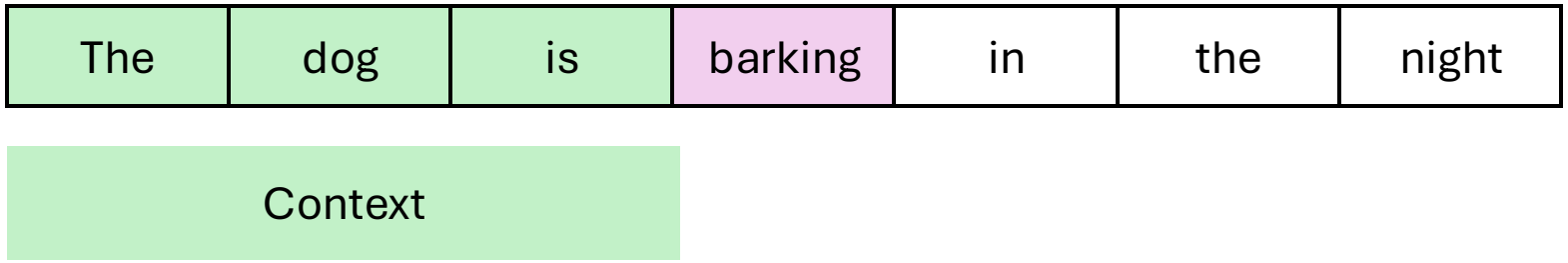
Learned embedding



Similarity(cat, furry) ~ 1
Similarity(cat, book) ~ 0

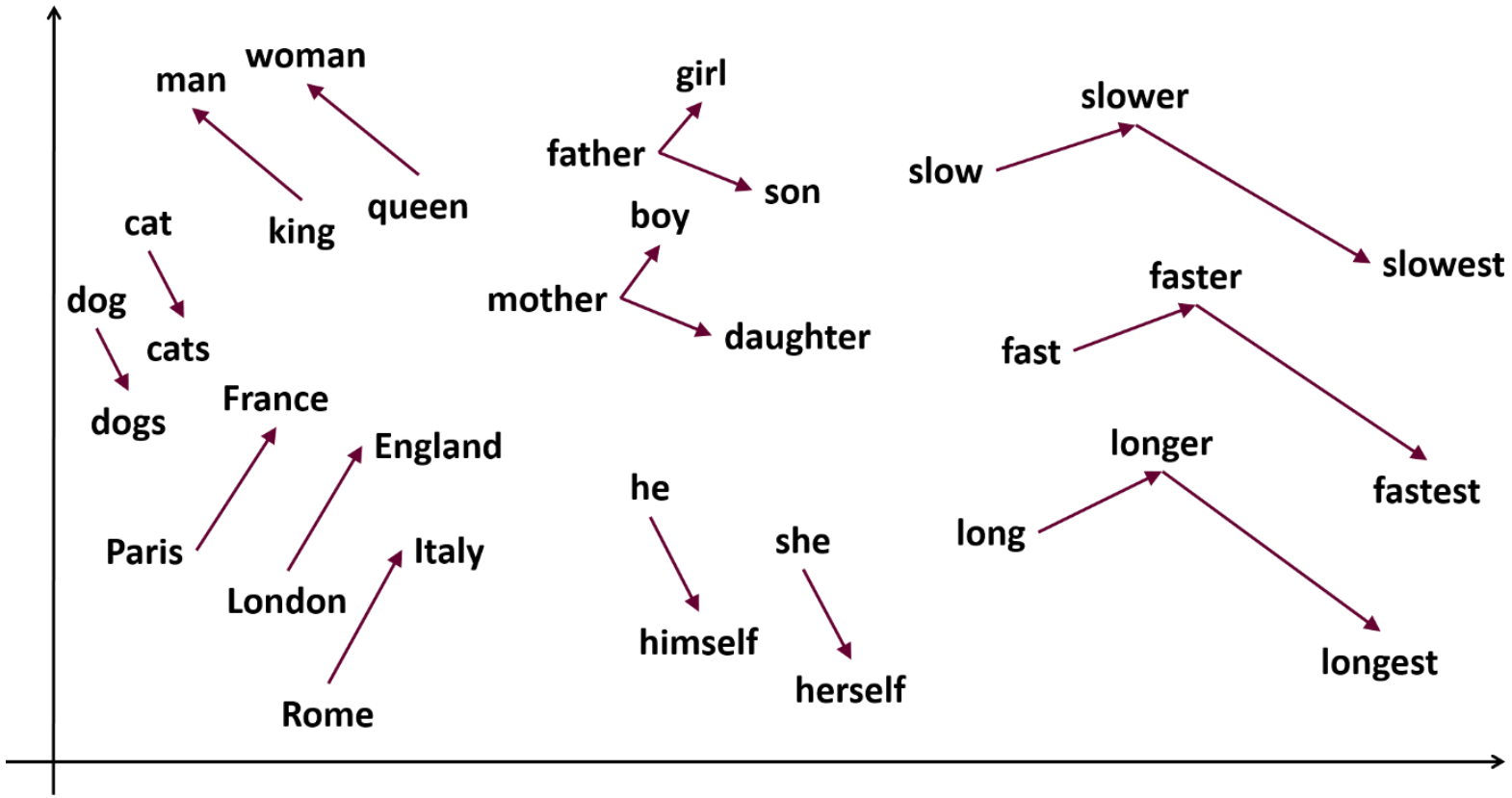
Embeddings

- A better way to convey meaning in the vectors itself



- Barking = [0.0012 0.1091 0.7386 -0.0105]
- Static vector
- Now we can measure similarity

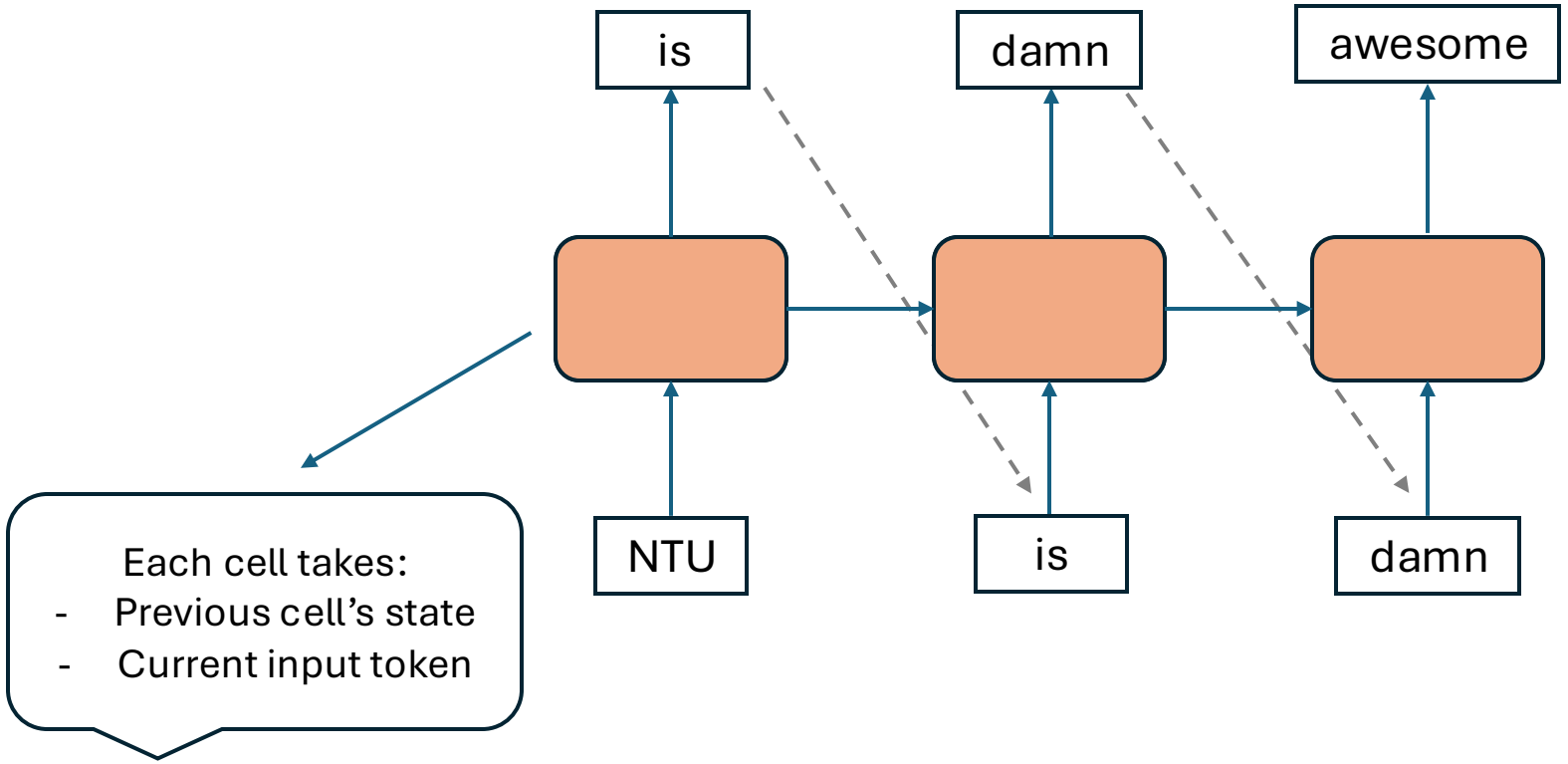
Word2Vec


man
woman
king
queen
girl
father
son
boy
mother
daughter
slow
slower
slowest
cat
cats
dog
dogs
faster
fast
fastest
France
England
Paris
London
Italy
Rome
he
she
himself
herself
long
longer
longest

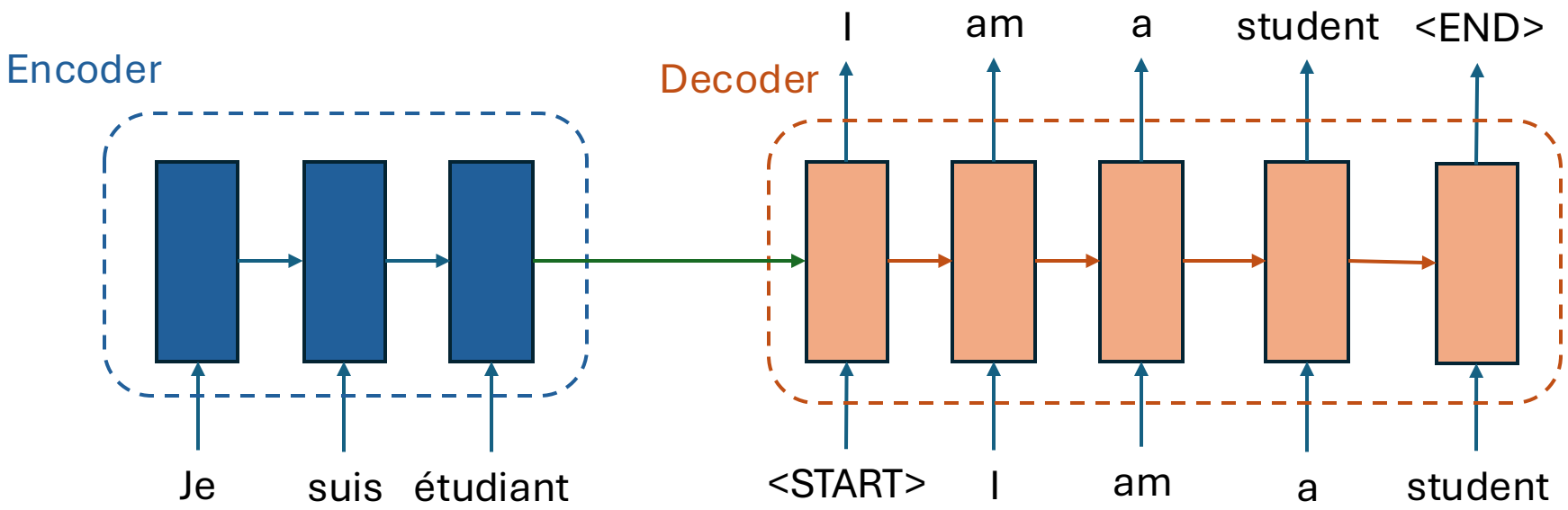
Model evolution

	1980s	Recurrent neural networks (RNNs)
Breakthrough!	1997	Long short-term memory (LSTM)
	2013	Word2Vec
Ground-breaking!!!	**2017**	**Transformer** 
  	**20s**	**Large language models (LLMs)** 

Recurrent neural network



Sequence to Sequence (Seq2Seq)



Machine translation using Seq2Seq

Limitations of RNNs

- Information bottleneck
- Slow, no parallelization
- Struggle to capture long-term memory

We want something better…

- ~~Information bottleneck~~ Continuous stream
- ~~Slow, no~~ parallelization
- ~~Struggle to~~ capture long-term memory

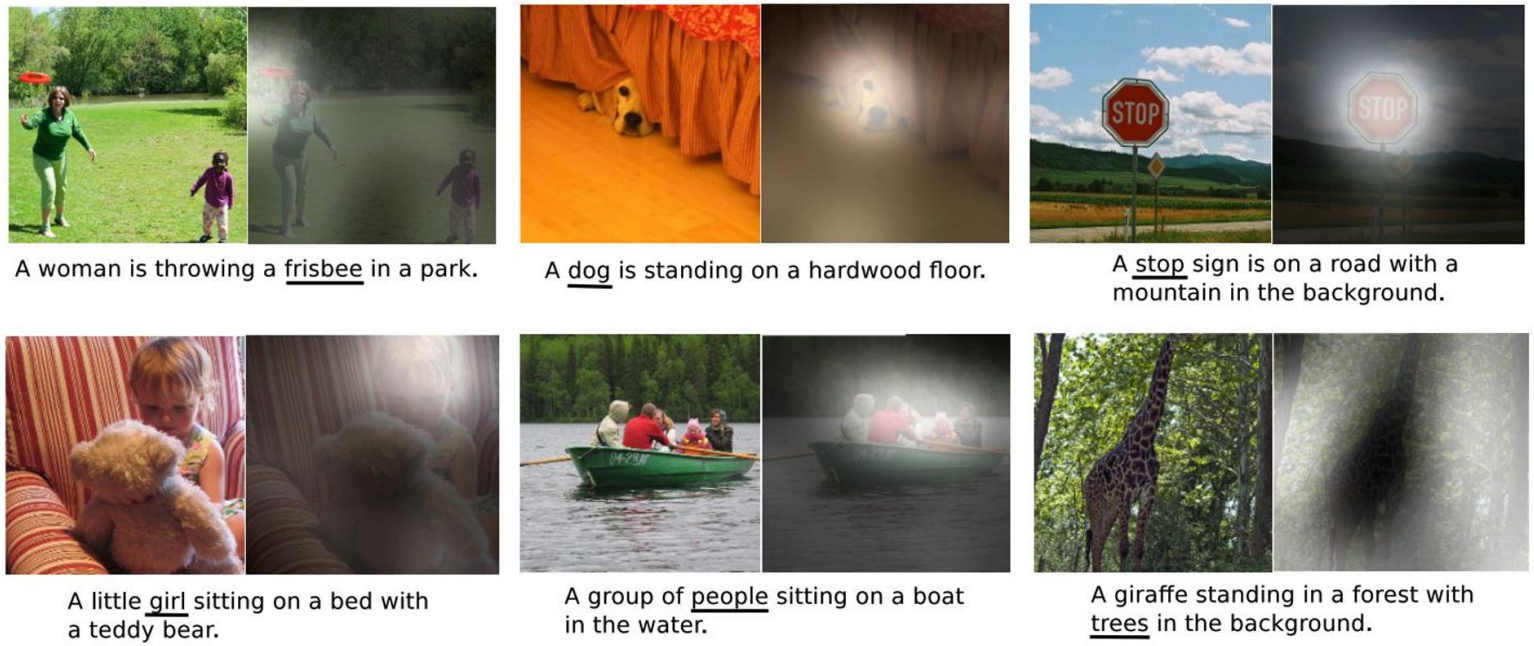
Attention is all you need

Attention is all you need

Attention is all you need

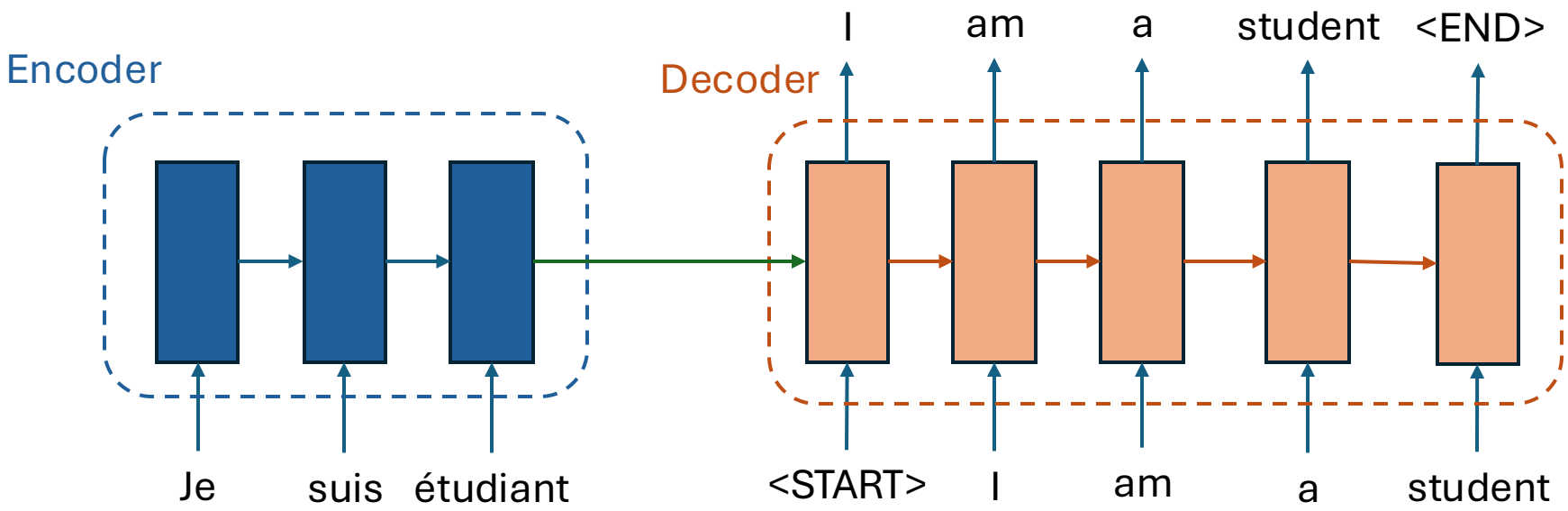
Attention is all you need

Image captioning with Attention



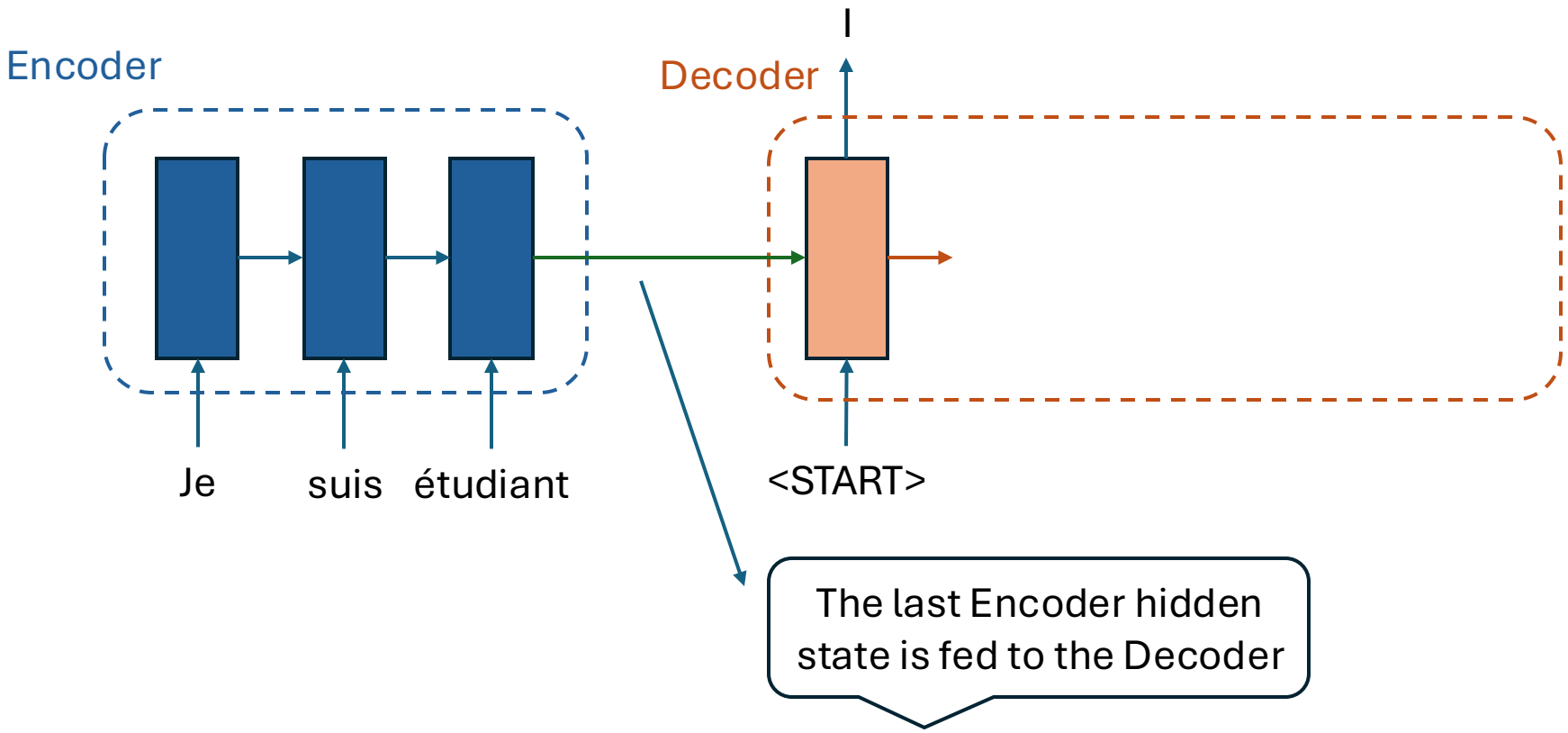
Attention is all you need

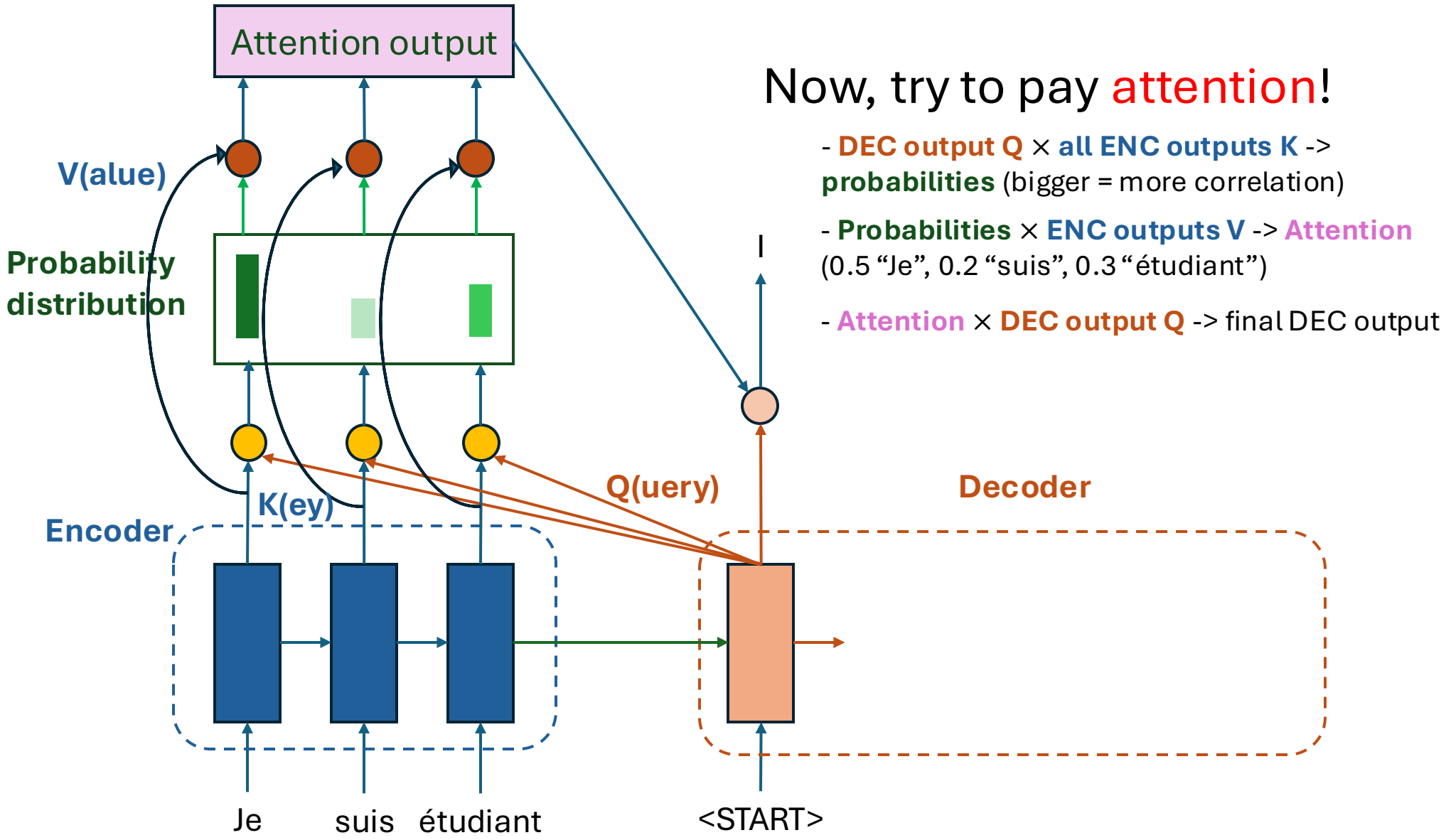
Revisit Seq2Seq recurrent network (no attention yet)



Attention is all you need

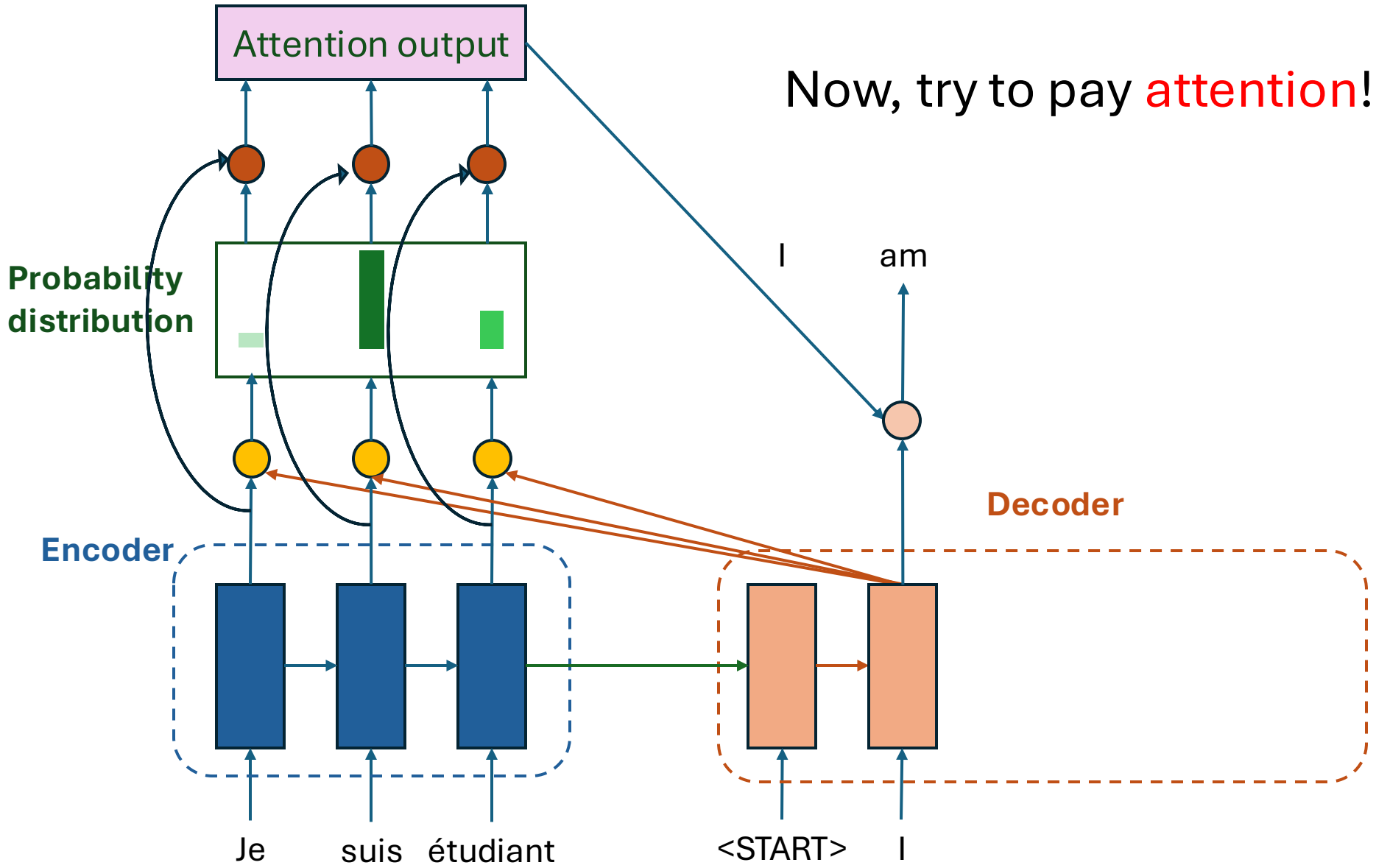
Looking at the first decoder time step

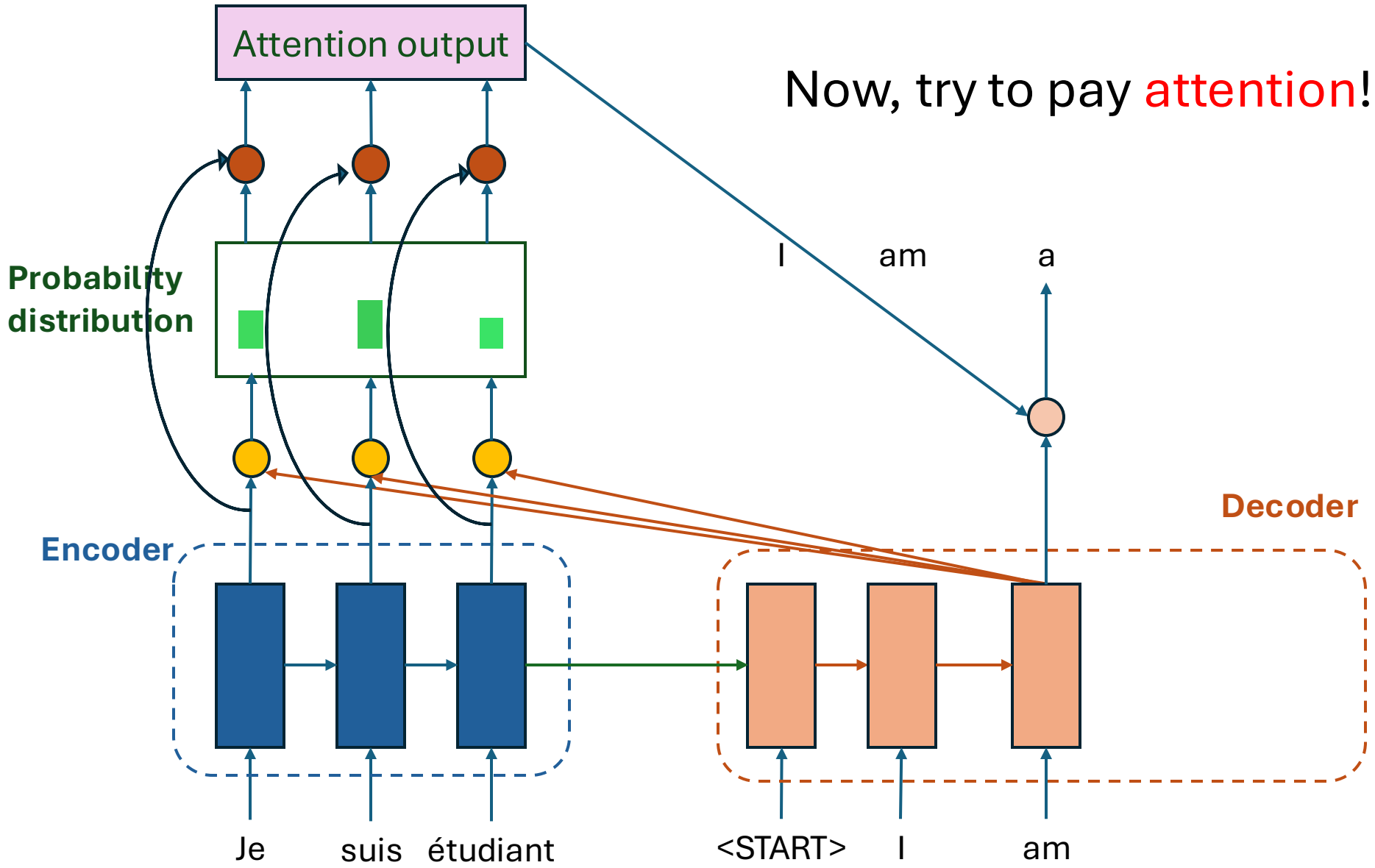




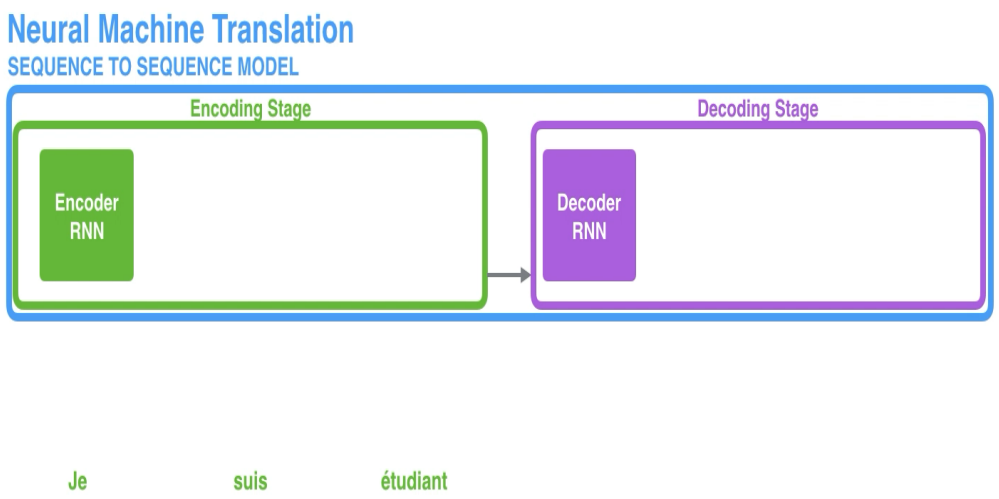
Now, try to pay attention!

- **DEC output Q** × **all ENC outputs K** -> **probabilities** (bigger = more correlation)
- **Probabilities** × **ENC outputs V** -> **Attention** (0.5 “Je”, 0.2 “suis”, 0.3 “étudiant”)
- **Attention** × **DEC output Q** -> final DEC output

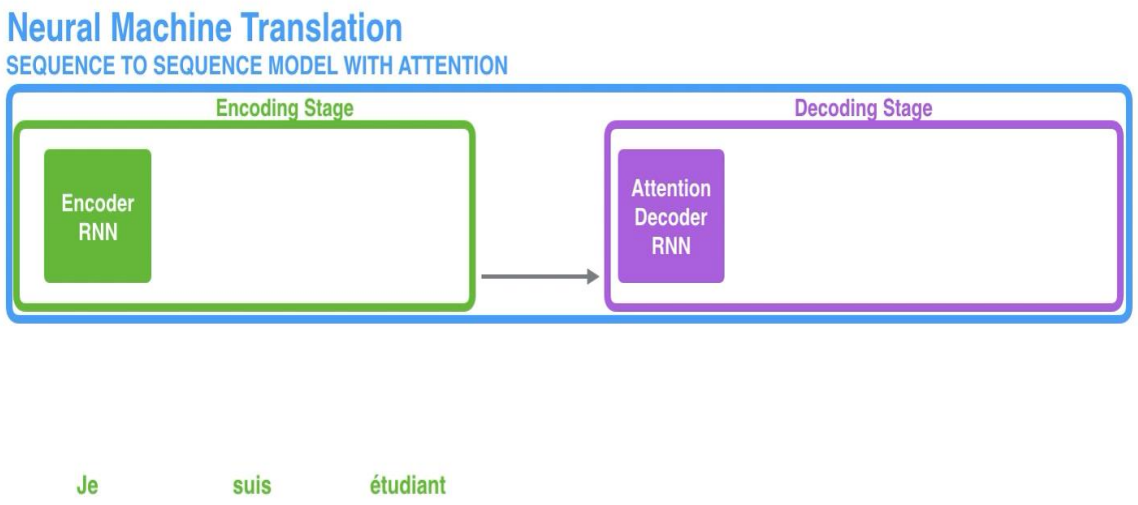

Attention output
Now, try to pay attention!
Probability distribution
I
am
Decoder
Encoder
Je
suis
étudiant
<START>
I


Attention output
Now, try to pay attention!
Probability distribution
I
am
a
Decoder
Encoder
Je
suis
étudiant
<START>
I
am

Attention is all you need



No Attention



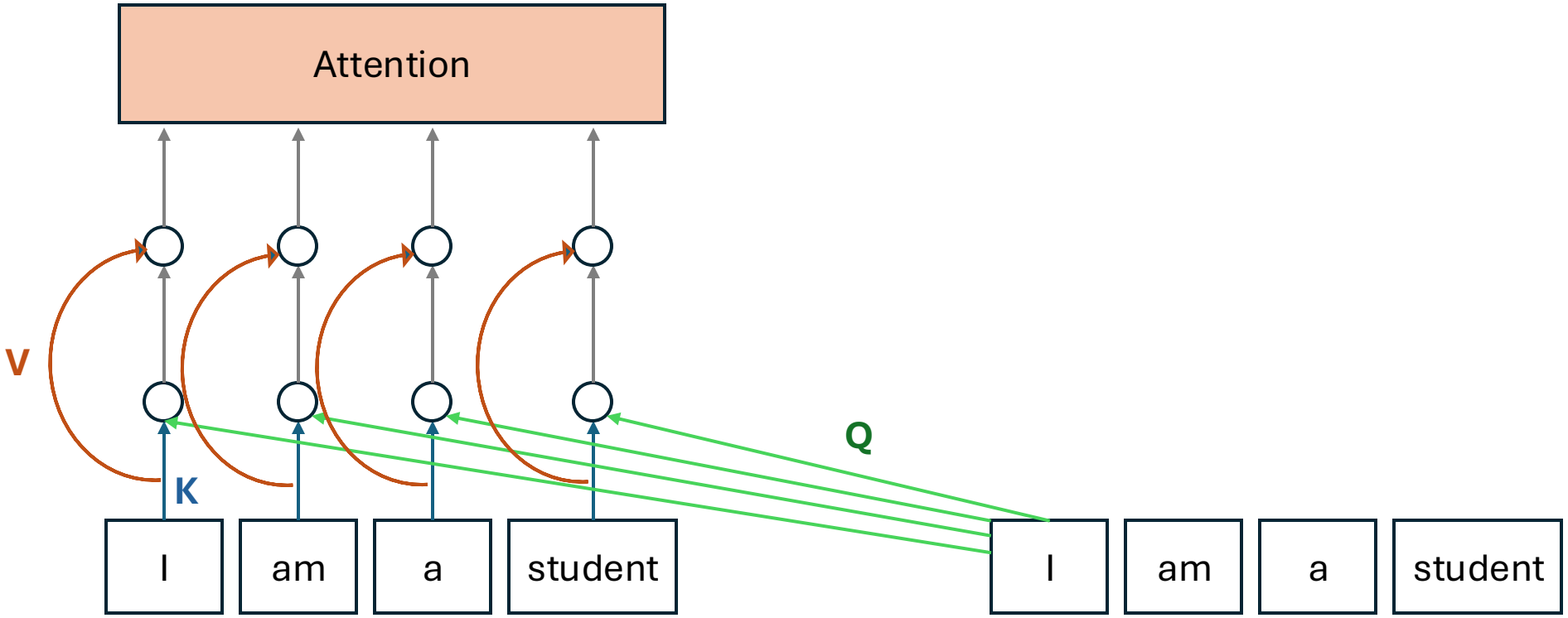
With Attention

Self-attention

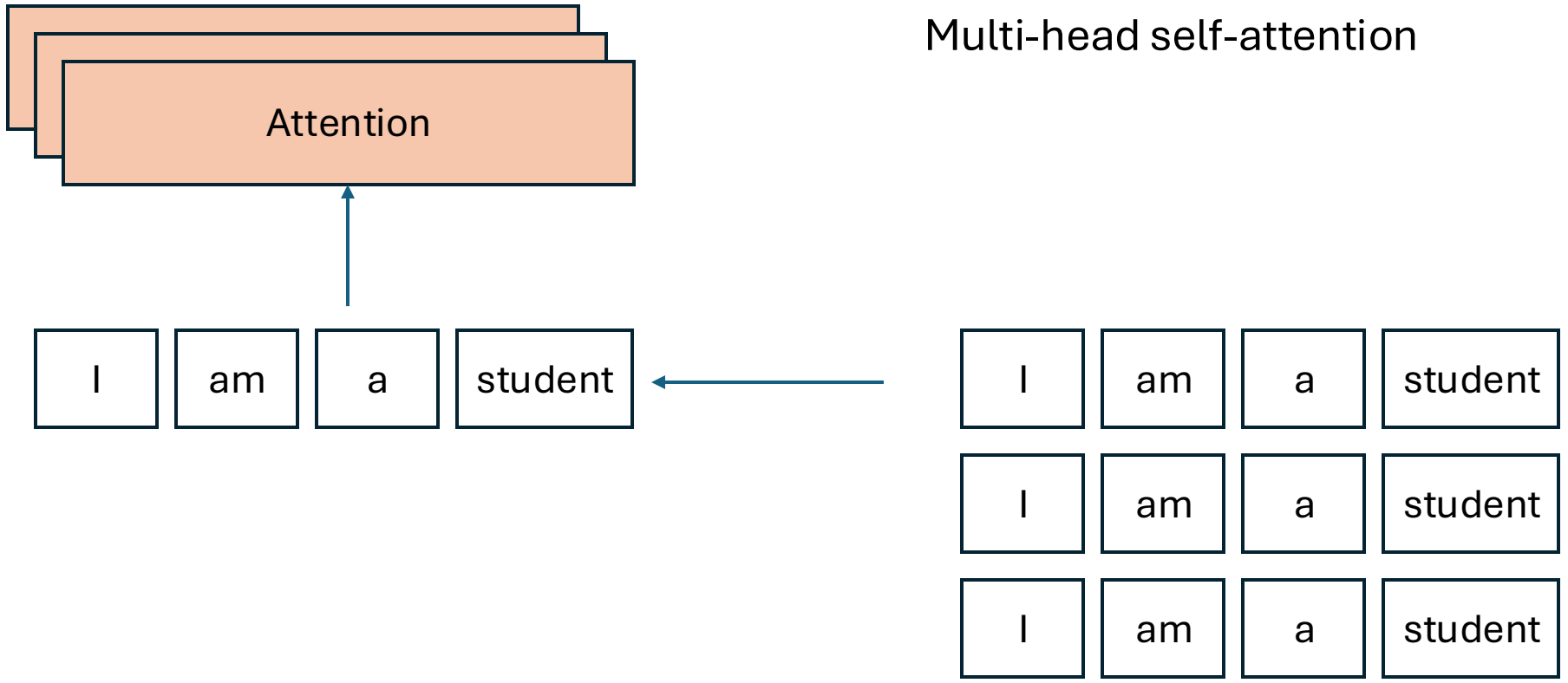
- Traditional attention: between encoder and decoder, good for sequence-to-sequence task such as machine translation.
- How about leveraging attention for learning representation?

-> ATTENTION with itSELF

Self-attention



Self-attention



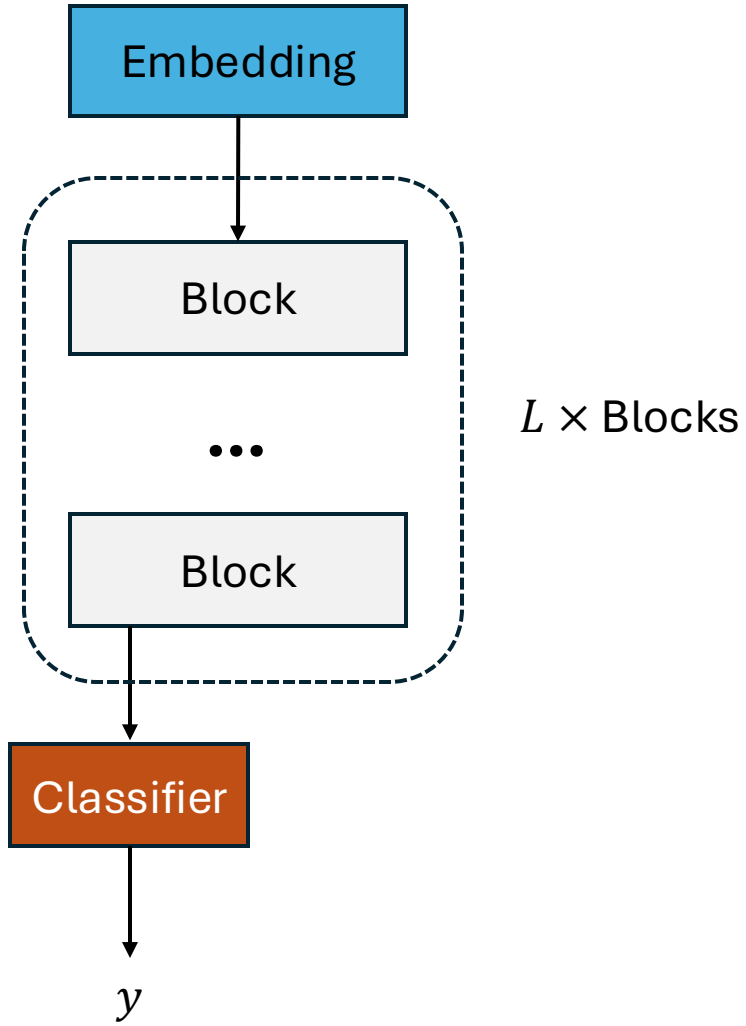
Multi-head self-attention

- Multiple heads expand the model’s capability to focus on different parts of the sentence.
- Improve the diversity of representation learning: each head tries to learn different representation subspaces of the input embeddings.

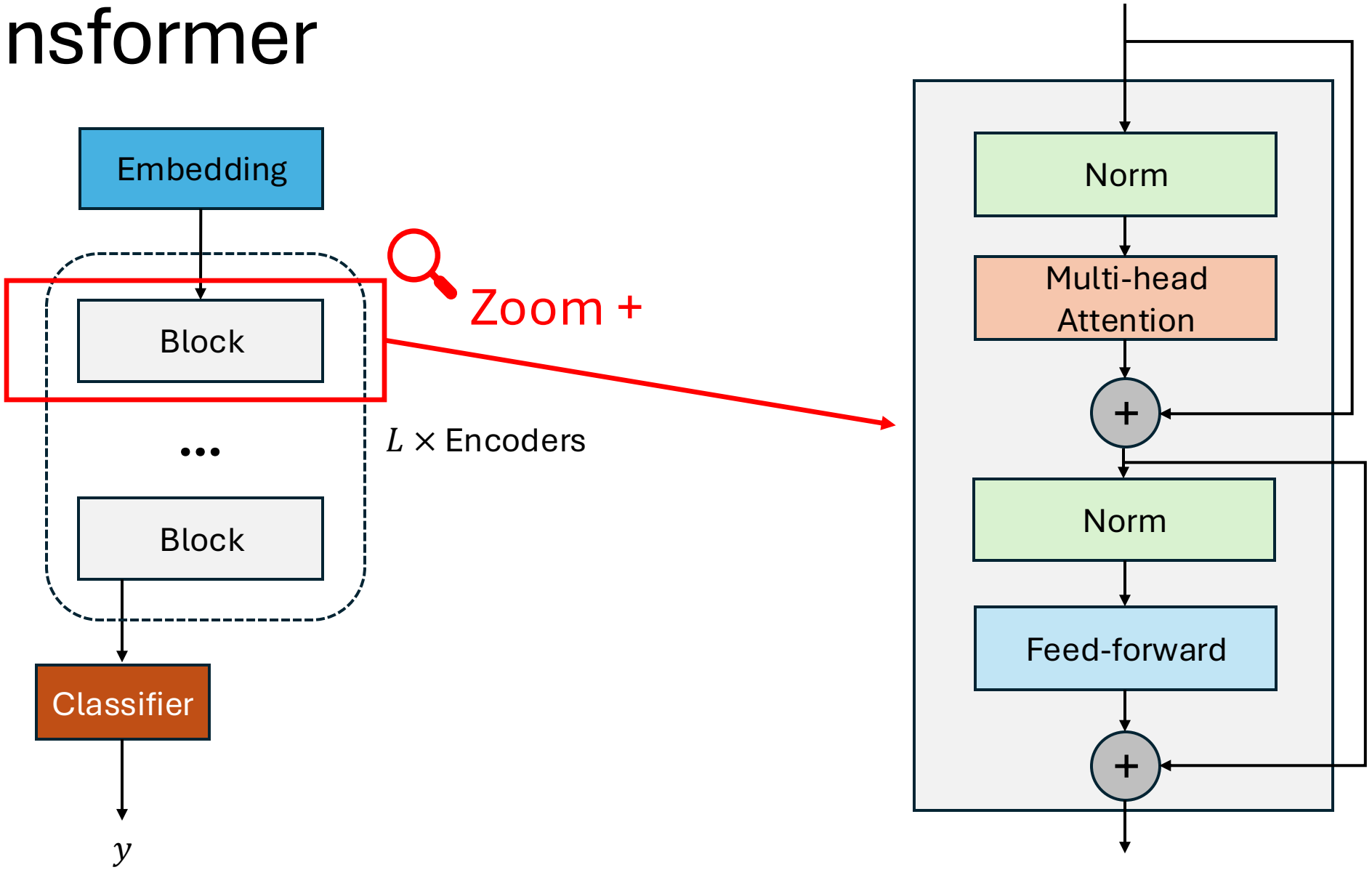
Are the issues resolved?

- ~~Information bottleneck :~~ attention takes all input positions into account.
- ~~Slow, no parallelization :~~ attention allows parallel computation (matrix multiplication).
- ~~Struggle to capture long-term memory :~~ also related to resolve information bottleneck.

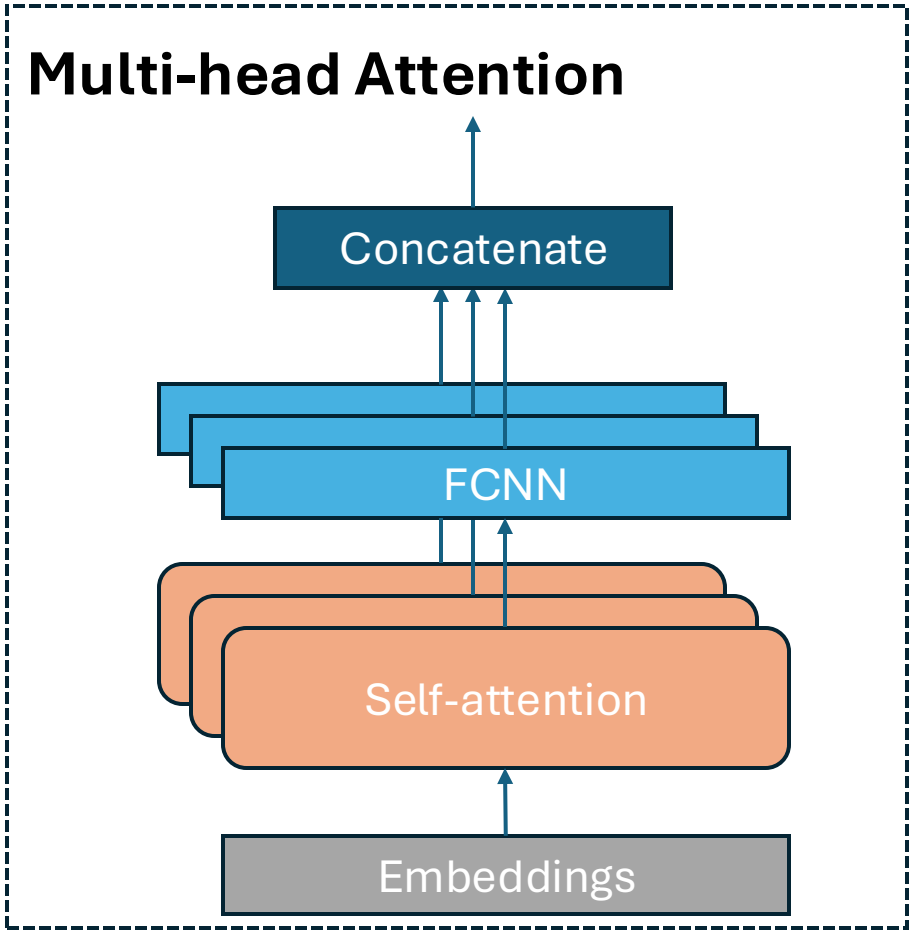
Transformer

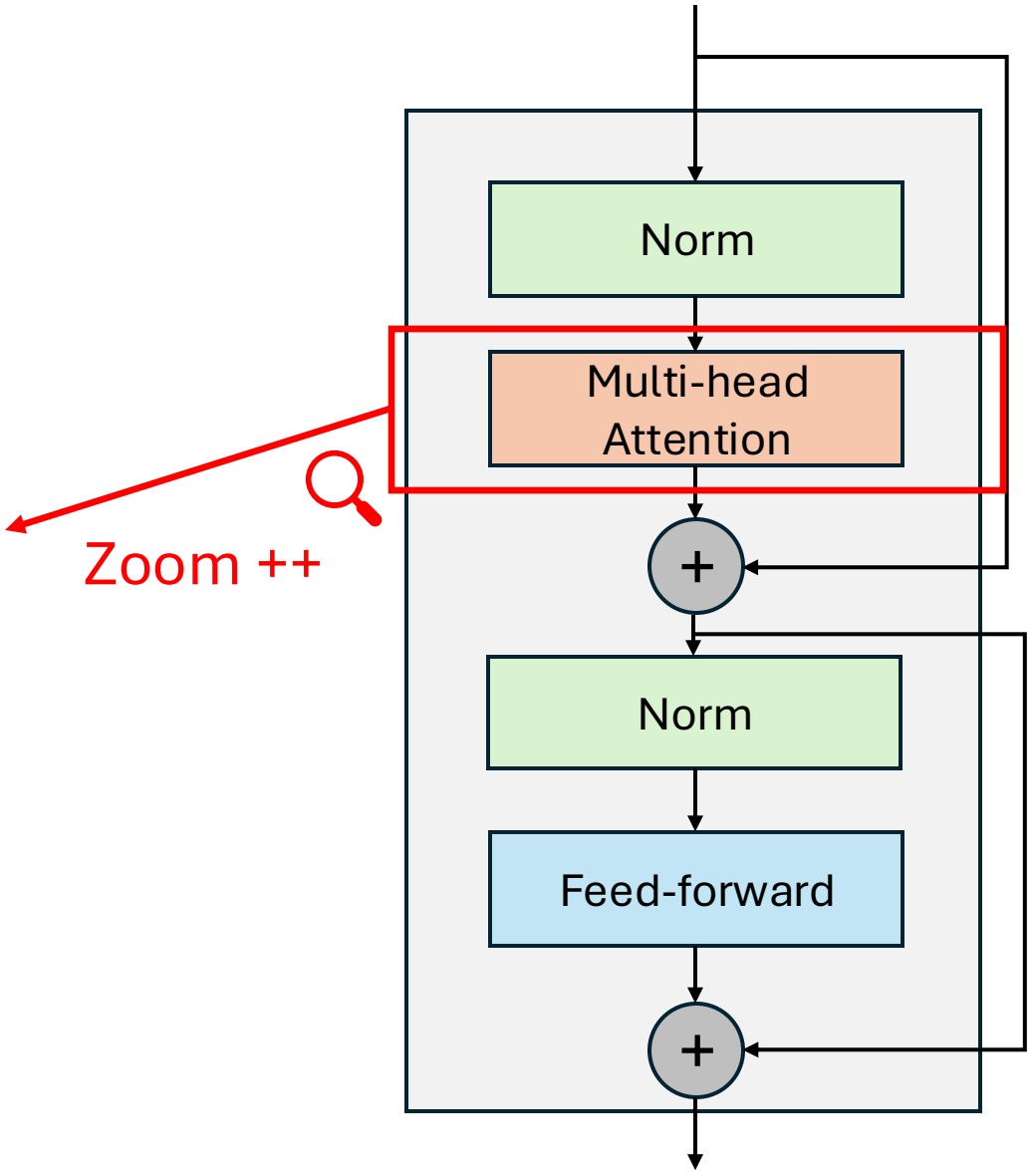


Transformer



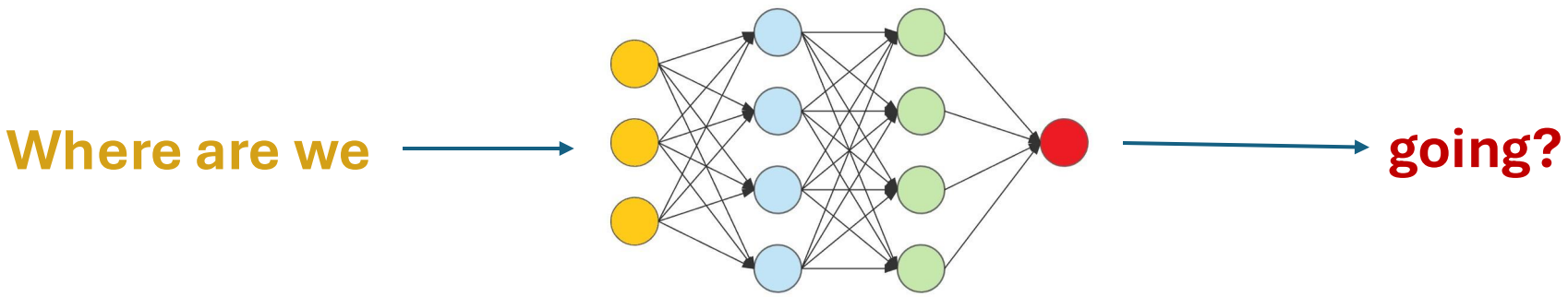
Transformer





Large language model (LLM)

Large language model (LLM)



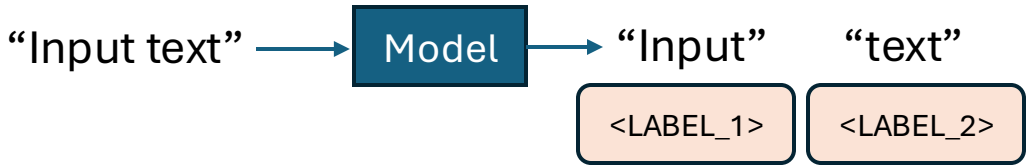
A language model

Task: next word prediction

NLP tasks (throwback)

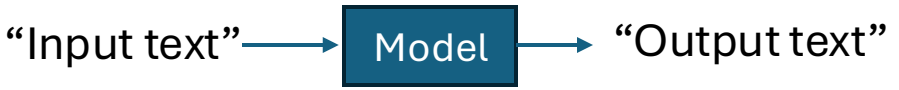
Classification

- Sentiment analysis + −
- Language detection GB IN CN DE ES
- Topic modeling Politics, Tech, Econ…



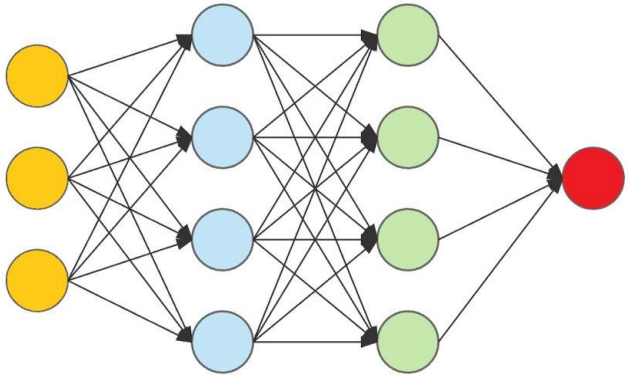
- Part of speech tagging VRB PRN NN ADJ
- Named entity recognition Person, Location, Time, …
- Dependency parsing

Generation

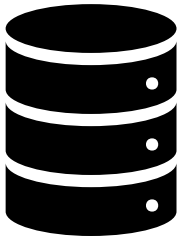


- Machine translation
- Question answering
- Summarization
- Text generation

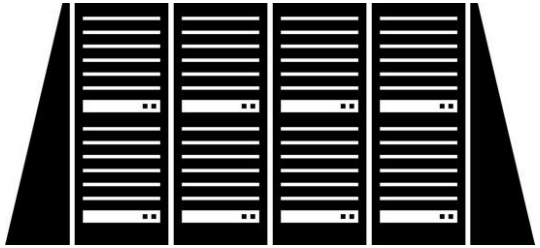
Large language model (LLM)



HUGE number of parameters



HUGE data



HUGE computing power

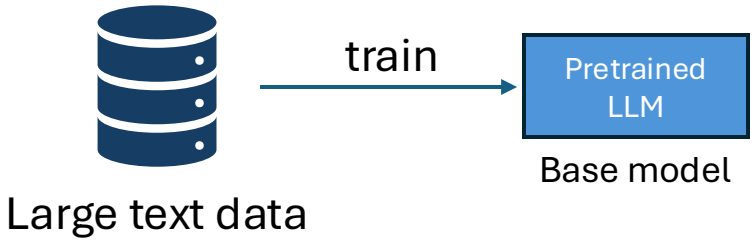
More capacity for other tasks

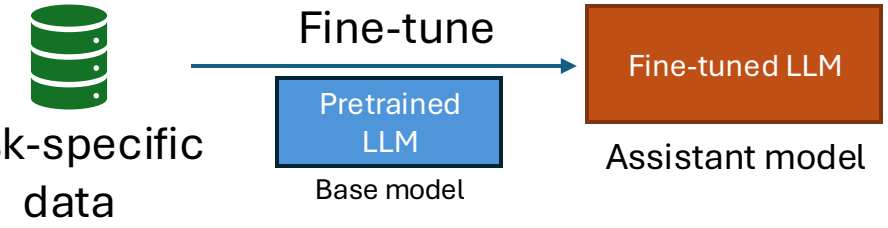
Large language model (LLM)

- Next word prediction is powerful! - LLMs are forced to learn tons of knowledge during training.

Alan Mathison Turing (/ˈtjʊərɪŋ/; 23 June 1912 – 7 June 1954) was an English mathematician, computer scientist, logician, cryptanalyst, philosopher and theoretical biologist.[5] He was highly influential in the development of theoretical computer science, providing a formalisation of the concepts of algorithm and computation with the Turing machine, which can be considered a model of a general-purpose computer.[6][7][8] Turing is widely considered to be the father of theoretical computer science.[9]

LLM – how to train?





Step 1: Pretrain on word prediction task -> base model (e.g. GPT-5)
- Using huge text corpus (no label)
- Costly
- Done once in a while

Step 2: fine-tune base model on downstream task -> assistant model
- Using LABELED task-specific dataset e.g. Q&A. Often smaller than text corpus.
- Much cheaper
- Fine-tune more regularly
- Assistant model will be deployed for public use (e.g. ChatGPT, DALL-E)

LLM – a bird’s eye view

https://bbycroft.net/llm

2 LLM families : BERT vs GPT

BERT (Encoder)

GPT (Decoder)

Next word prediction

- Given context predict center word

The	dog	barks	when	I	?			

The	dog	barks	when	I	?	on	the	door

Bidirectional Embeddings

The	dog	barks	when	I	knock	on	the	door

The	tree	bark	is	rough	when	I	touch	it

- Different embeddings for different contexts

BERT is a model

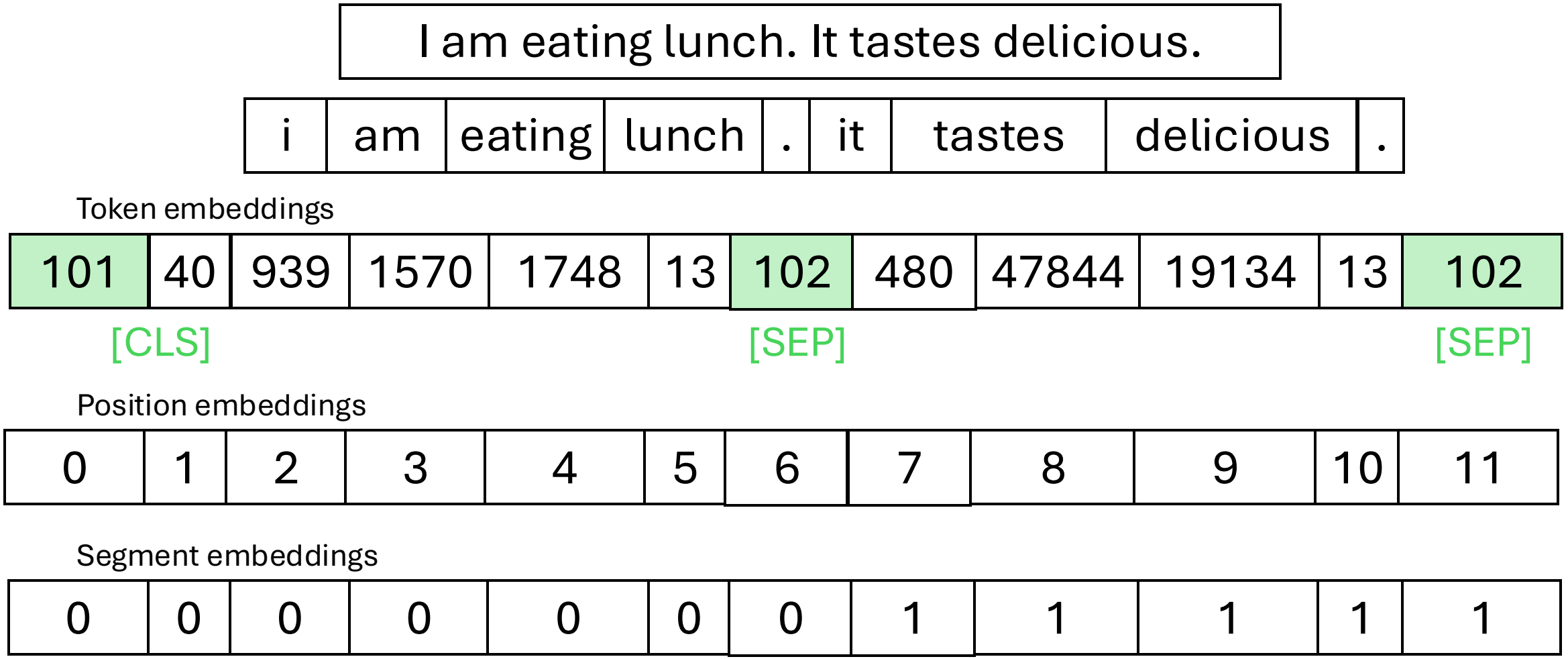
- The embeddings are as good as the training
- BERT's primary objective was Masked Language Modeling

  I am eating lunch. It tastes [MASK].

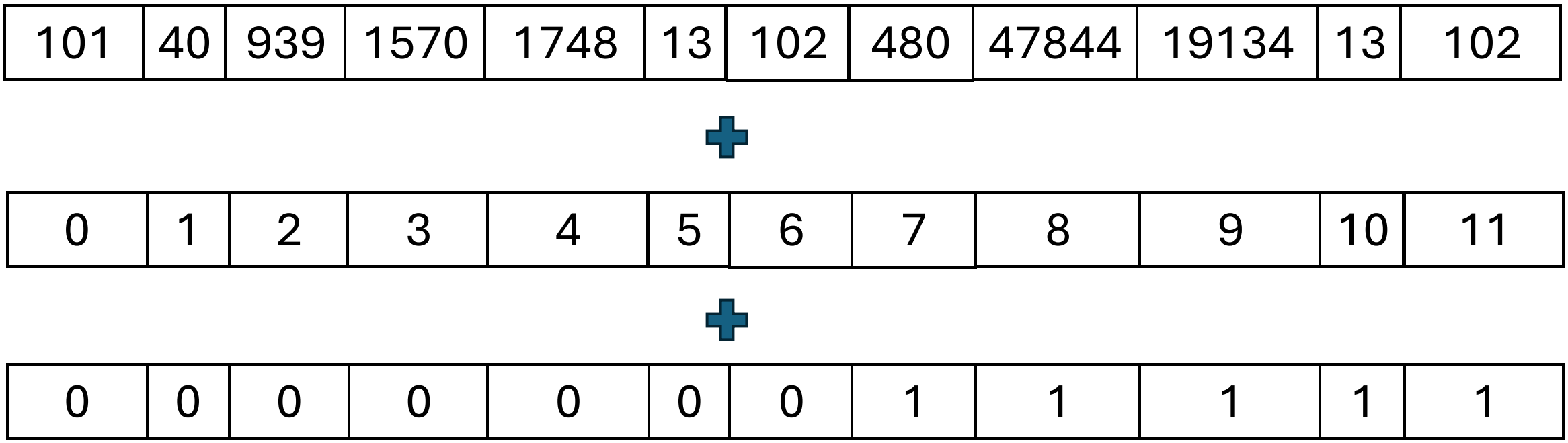
- BERT's secondary objective was Next Sentence Prediction

  I am eating lunch. [???]

BERT tokens


I am eating lunch. It tastes delicious.
i
am
eating
lunch
.
it
tastes
delicious
.
Token embeddings
101
40
939
1570
1748
13
102
480
47844
19134
13
102
[CLS]
[SEP]
[SEP]
Position embeddings
0
1
2
3
4
5
6
7
8
9
10
11
Segment embeddings
0
0
0
0
0
0
0
1
1
1
1
1

BERT embedding



Different BERT models

- Multilingual BERT
- MedBERT- medical LM understands health records, diseases
- LegalBERT – law LM understands legalese, patents
- PolyBERT – chemical LM

GPT

- Both BERT and GPT are transformer-based architectures
- GPT is a generative model trained for Next Word Prediction
- Great at generation, summarization, translation

2 LLM families : BERT vs GPT

BERT (Encoder)

Reads both directions

Sees the full sentence at once — left AND right context.

Good at

Understanding text: classification, extraction, Q&A.

Training method

Masked Language Modelling — randomly hide words and predict them.

GPT (Decoder)

Reads left to right only

Predicts one token at a time — cannot look ahead.

Good at

Generating text: writing, summarizing, answering, reasoning.

Training method

Next Token Prediction — predict each word given all prior words.

Are embeddings only for text?

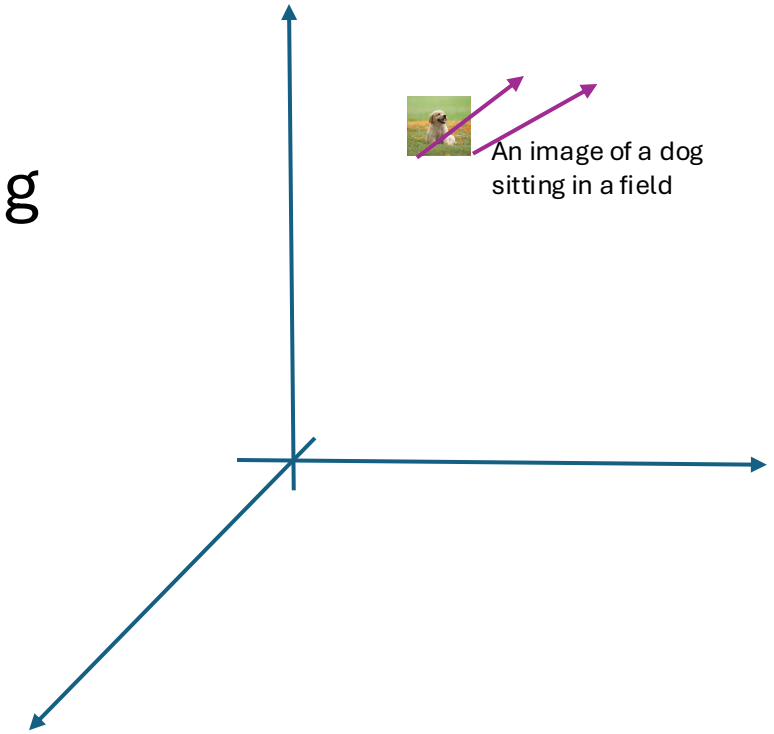
- Sentiment Analysis, Spam detection, Topic modeling
- Named Entity Recognition
- Text generation – autocomplete, summarization
- Machine Translation
- Information Retrieval, question answering
- Image captioning
- Image generation from prompt

MULTI - MODAL

CLIP embeddings

+

An image of a dog sitting in a field



How can I use CLIP?

Task: Classification

TRAINING

Image + CLIP caption

A photo of a {class label}

INFERENCE

✓ a photo of a **television studio**.

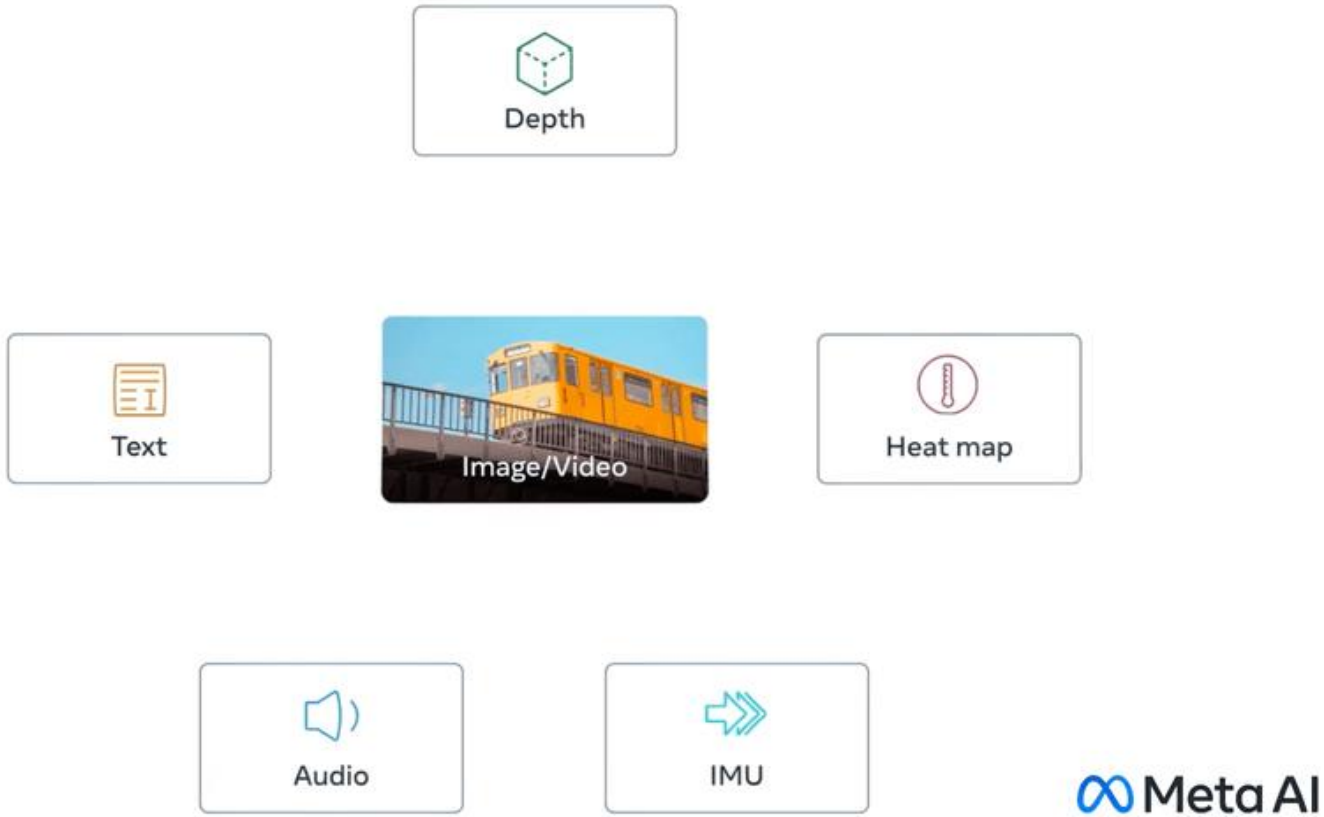
× a photo of a **podium indoor**.

× a photo of a **conference room**.

× a photo of a **lecture room**.

× a photo of a **control room**.

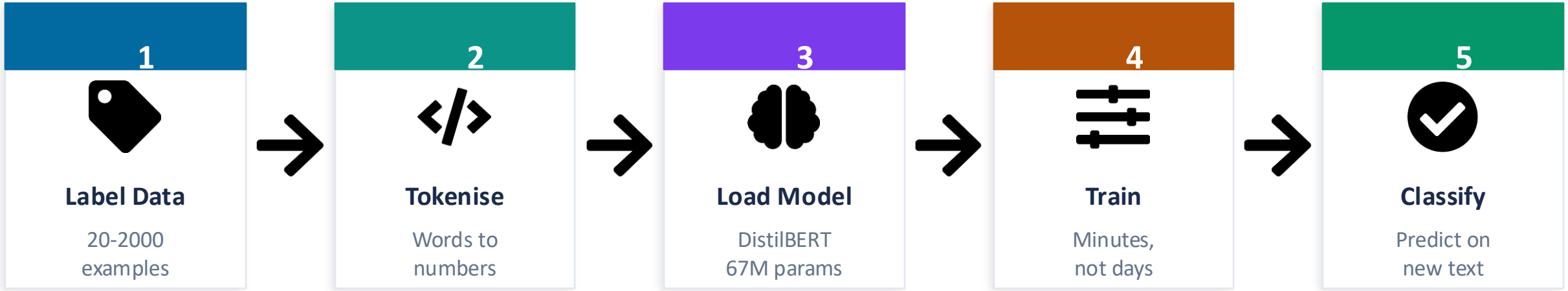
Multi-modal embeddings



Practical uses of LLMs

- Fine-tuning on domain-specific data
- Prompt engineering
- Connect LLMs to your documents - RAG

Fine-tuning


1
Label Data
20-2000 examples
2
Tokenise
Words to numbers
3
Load Model
DistilBERT 67M params
4
Train
Minutes, not days
5
Classify
Predict on new text

Prompt Engineering

A prompt is an instruction fed to the model as input text.

Because the model was trained on billions of examples of humans answering questions and summarizing text, it has learned to pattern-match your instruction to a likely good response.

Training data exposure

Context window

In-context learning

Zero-shot Prompting

- You give the model a task with no examples.
- It still works because it *generalizes* from its vast training data.
- Zero-shot fails on custom label schemes the model has never seen.

Few-shot Prompting

- Provide 2-3 examples of input-output before your real question. The model figures out the pattern on the fly —no training needed.
- Examples teach format and intent, in immediate context window – which shifts the probability.
- More examples are not better!

Retrieval Augmented Generation

LLMs have a knowledge cutoff and no access to your documents.

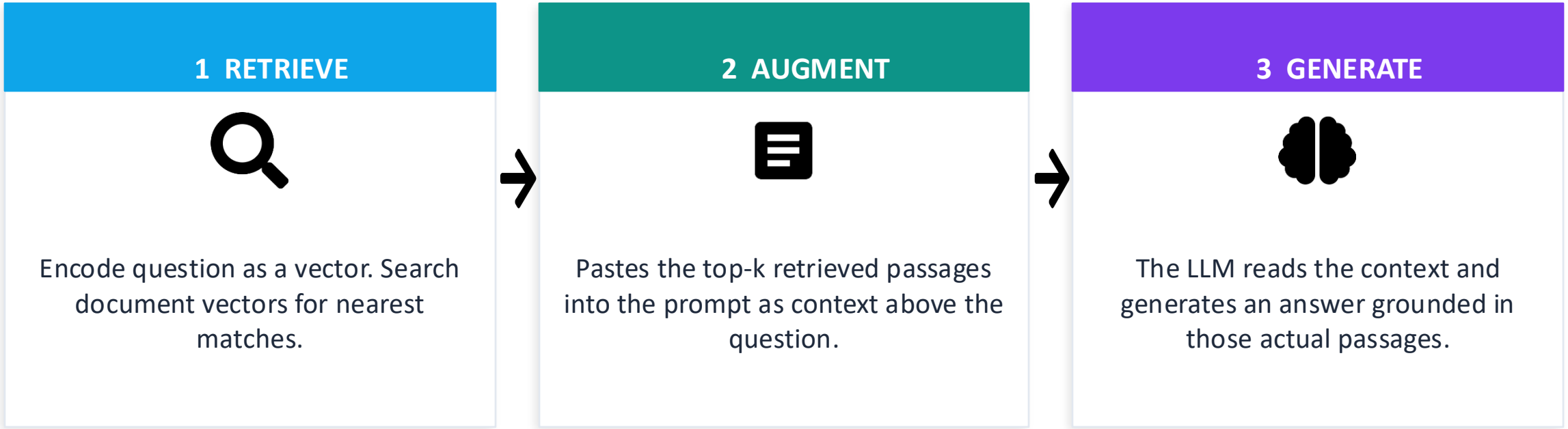
If you ask GPT-4 about your internal lab report, last month's paper, or a proprietary dataset, it cannot answer accurately. It will either say it does not know, or confidently make something up. #hallucination.

RAG: give the model access to your documents at query time.

Instead of relying on training memory, the model receives relevant passages from your documents directly in the prompt. It reads them and answers from that text.

- Less hallucination
- No retraining needed
- Data stays private
- Documents can change

RAG


1 RETRIEVE
Encode question as a vector. Search document vectors for nearest matches.
2 AUGMENT
Pastes the top-k retrieved passages into the prompt as context above the question.
3 GENERATE
The LLM reads the context and generates an answer grounded in those actual passages.

Domain Application Example Walkthrough

3rd DL Bootcamp (2025-26) - Session 4 - Post Session Feedback